Visualization of β-reduction Graphs in the Lambda Calculus

Niels Bjørn Bugge Grathwohl og Jens Duelund Pallesen

13. august 2010

Indhold

- Introduktion
- Design
  - Grafgenerator
  - Graftegner
  - GUI
- Eksempler
  - Predecessor og Successor
  - Addition, multiplikation og potens
  - Hypercube
  - Association
  - Cons
- Performance
- Fremtidigt arbejde
- Konklusion

Introduktion

- Der findes visualiseringer af lambda kalkylen generelt, men ingen visualiseringer af reduktionsgrafer.
- Disse visualiseringer er forskellige måder at tegne syntakstræer for en term på, på forskellige stadier i en kæde af reduktioner af den term.
- Programmet udvilket i dette projekt illustrerer de forhold der gælder mellem termer ved β-reduktion, ved at lægge dem ud som en graf.
- Produktet er udviklet med henblik på forskere og undervisere inden for området.

Design

- Opdelt design, gør det muligt at udskifte individuelle dele uafhængigt af hinanden.
- Designet gør det let at udvide med nye tegnealgoritmer.
- Multiplatform (Python og GTK).

Design: Grafgenerator

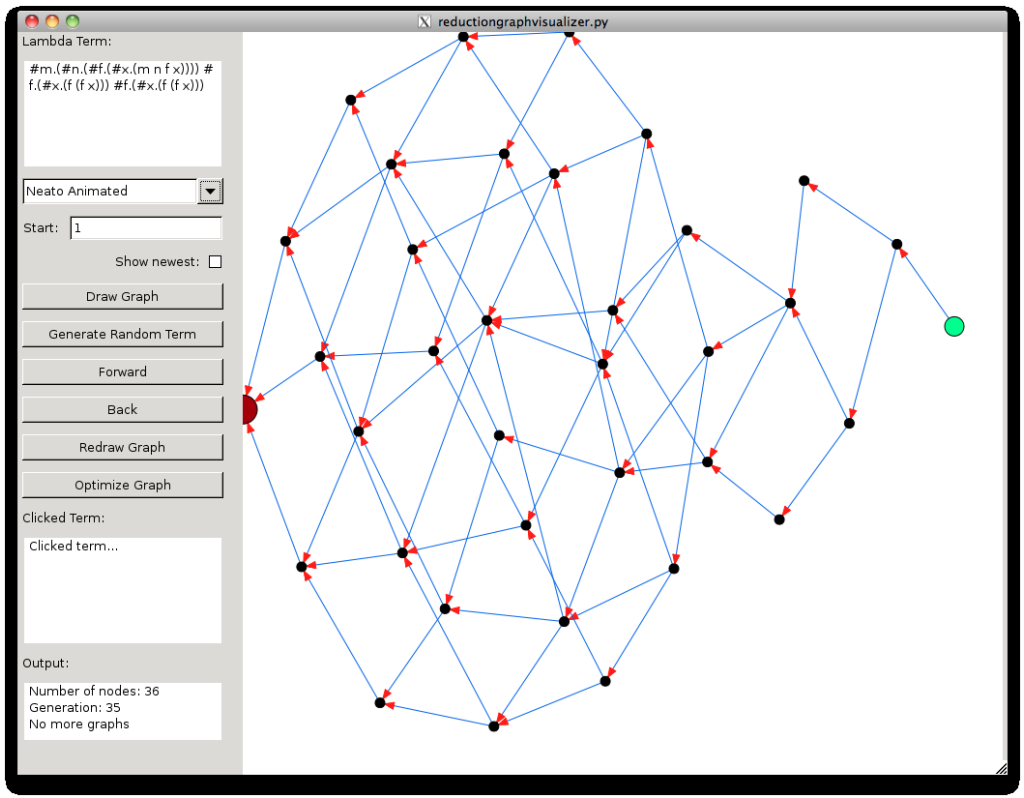
- Parse $\rightarrow$ Find redexer $\leftrightarrow$ Reducér
- Parseren oversætter en tekststreng til en interne DAG-repræsentation af en term.
- På denne repræsentation anvendes en funktion der returnerer alle positioner på redexer i termen.
- Denne liste løbes igennem, og redexerne reduceres til deres kontrakta.
- Hvert uset kontraktum tilføjes reduktionsgrafen.
- Forbindelser mellem termen der reduceres samt dets kontrakta indkodes i reduktionsgrafen, og næste term i grafen tages under behandling.

Design: Graftegner

- GraphViz – Tilbyder en række tegnealgoritmer.
- Majorization – Vores egen implementation af en tegnealgoritme, som tilbyder ekstra fleksibilitet i forhold til GraphViz's tegnealgoritmer (animering, tweaks).
- PyGTK – Brugt til GUI pga. god understøttelse samt at det er enkelt at benytte.
- Python Widgets – Brugt til at binde de forskellige GUI elementer sammen.

Design: GUI

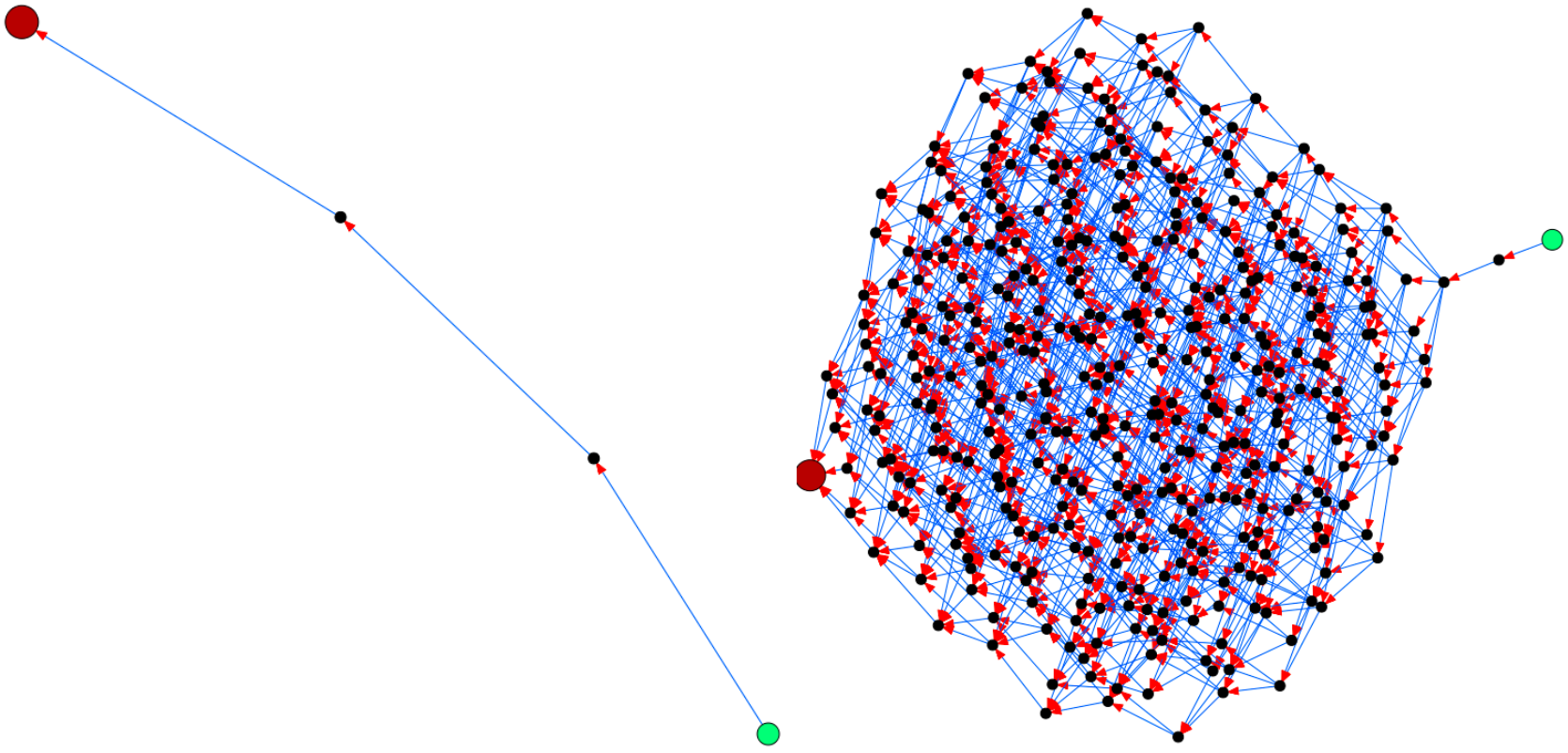
- Ikke en GUI opgave.
- Dog er et minimum af GUI en nødvendighed for at kunne anvende programmet
- Enkel GUI



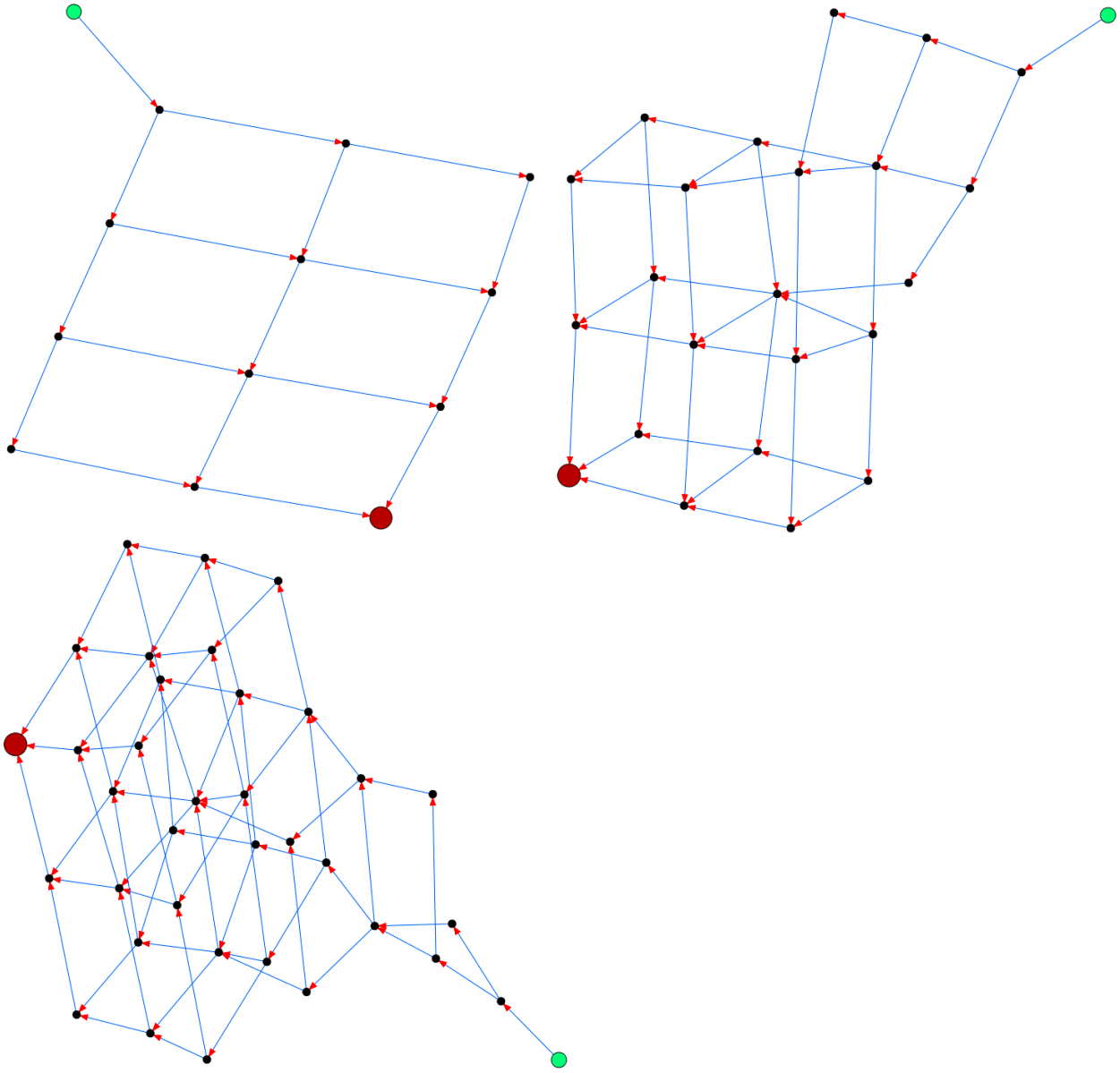
Eksempler

- Predecessor og Successor.
- Addition, multiplikation og potens.
- Hypercube.
- Association.
- Cons.

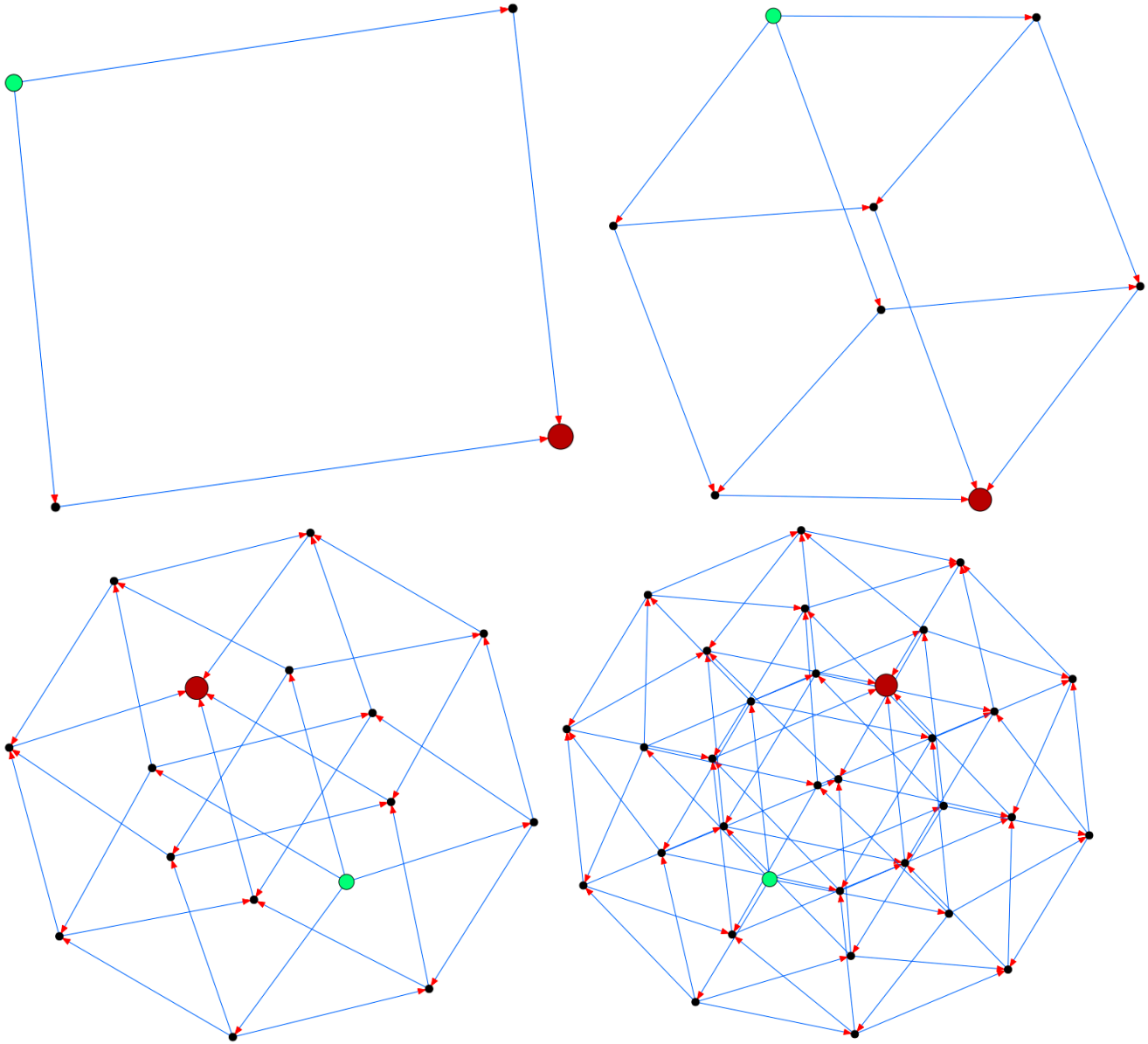
Predecessor og Successor



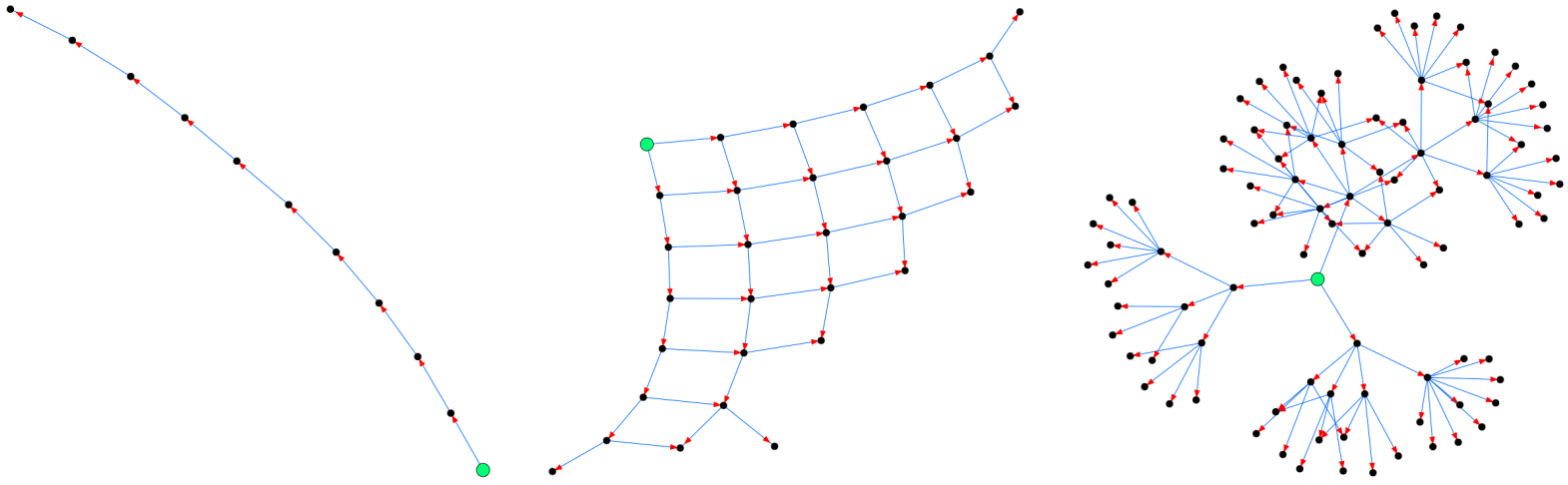
Addition, multiplikation og potens



Hypercube $(((\lambda x_1.x_1)f)\ldots((\lambda x_n.x_n)f))$

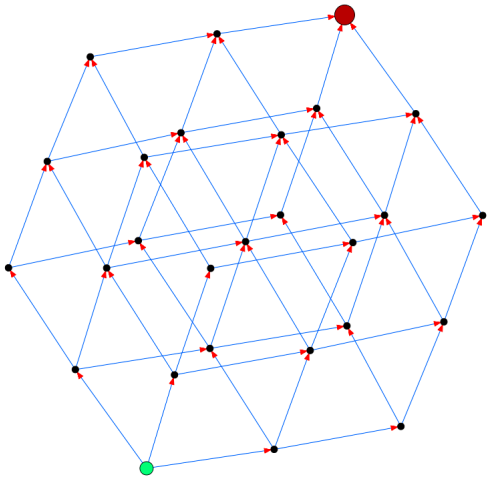


Association

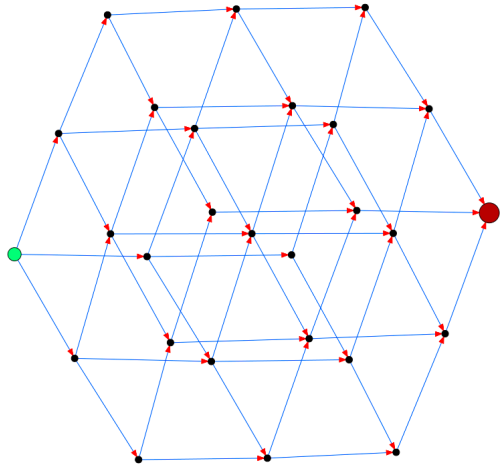


(a) $\omega_3\omega_3\omega_3\omega_3$ (b) $\omega_3\omega_3(\omega_3\omega_3)$ (c) $\omega_3(\omega_3(\omega_3\omega_3))$

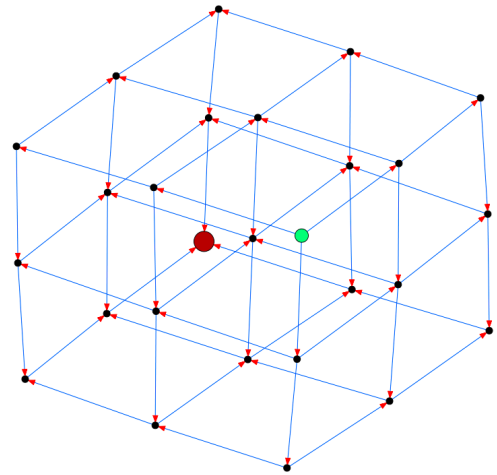
Cons



(d) (2,3)



(e) (4,4)



(f) (5,0)

Performance

- Neato giver bedste all-round resultater mht. brugbarhed.
- Benchmarks (kørsler af layoutalgoritmerne uden GUI og uden at tegne grafen):

Lambda Term: Predecessor til 4 Generationsnummer: 300					
Neato Animated	Circo	Dot	Neato	TwoPi	Fdp
4.36 s	310.44 s	0.91 s	1.11 s	0.23 s	10.90 s

Lambda Term: 2^2 Generationsnummer: 35					
Neato Animated	Circo	Dot	Neato	TwoPi	Fdp
0.05 s	0.12 s	0.04 s	0.05 s	0.04 s	0.09 s

Fremtidigt arbejde

- Udbygge med flere graftegnealgoritmer.
- Udbygge med flere reduktionssystemer end lambda kalkyle.
- OpenGL omskrivning (3D, zooming, etc.)
- Highlighting af forskellige reduktionsstrategiers “stier” gennem reduktionsgrafen.
- Multitrådet.
- Eksport/import af grafer.
- Reel brugertest.

Konklusion

- En 1. version der gør det muligt hurtigt at se komplekse reduktionsgrafer, der ville tage lang tid at udregne og tegne i hånden.
- Gode udbyggelsesmuligheder.